



Asignatura:

Arquitectura y Organización
de Computadores

Documentación:
Práctica Conecta 4

PROFESORES

José Manuel Sánchez Mañes
Daniel León González

GRUPO: LA FAMILIA TRAPISONDA

Víctor Manuel Sanz Rubio
Roberto García Ramírez
Ignacio González de la Cruz

Índice

1	Introducción	2
1.1	Objetivo de la práctica	2
1.2	Descripción	2
2	Flujo de rutinas y ejecución	4
2.1	Resumen general.....	4
3	Rutinas principales.....	10
3.1	Listado de rutinas y descripción.....	10
4	Manual de uso.....	11
4.1	Requisitos previos.....	11
4.2	Como iniciar el juego	11
4.3	Mecánicas del juego	11
5	Porcentajes de participación	13
6	Bibliografía	14

Índice de figuras

Figura 1-1: Estructura del directorio de la práctica	3
Figura 2-1: Flujo principal del programa.....	4
Figura 2-2: Archivo ConnectFour.asm	4
Figura 2-3: Pantalla StartScreen.asm	5
Figura 2-4: Pantalla GameScreen.asm	6
Figura 2-5: Pantalla EndScreen.asm: Jugador ganador	7
Figura 2-6: Pantalla EndScreen.asm: Empate entre jugadores	8
Figura 2-7: Pantalla common.asm: Mensaje de ¡Hasta pronto!.....	9

1 Introducción

1.1 Objetivo de la práctica

El objetivo principal de este proyecto es el desarrollo de una implementación completa del juego clásico "Conecta 4" para la plataforma ZX Spectrum 48K. La práctica abarca la gestión de gráficos de bajo nivel (píxeles y atributos), la lógica de juego, el control de estado y el manejo de entradas de teclado en ensamblador Z80.

1.2 Descripción

Este proyecto es una recreación del juego de estrategia para dos jugadores "Conecta 4". Los jugadores (Rojo y Amarillo) se turnan para soltar fichas en una cuadrícula vertical de 6 filas por 7 columnas. El primer jugador que consiga alinear cuatro de sus fichas en horizontal, vertical o diagonal, gana la partida.

El programa está escrito íntegramente en ensamblador Z80 y está estructurado en múltiples archivos modulares que gestionan la lógica, los gráficos y las pantallas. Gestiona el movimiento (Q/W para el Jugador 1, O/P para el jugador 2) con un temporizador de 16 bits y espera la pulsación de ENTER.

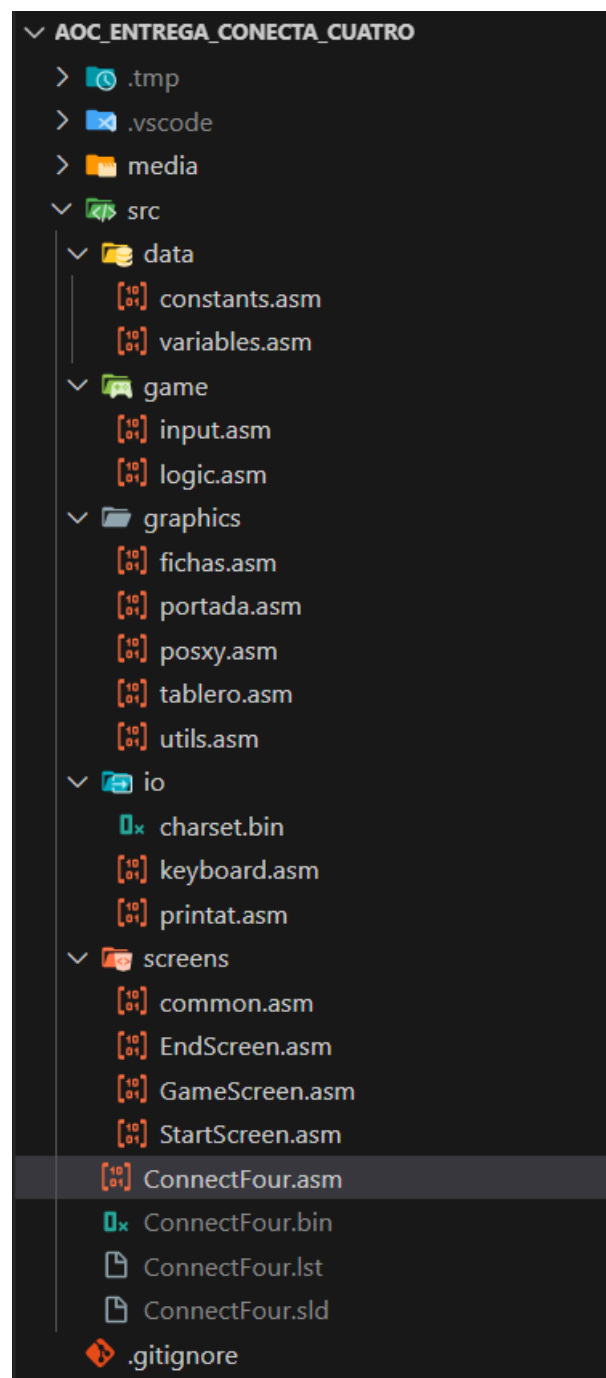


Figura 1-1: Estructura del directorio de la práctica

2 Flujo de rutinas y ejecución

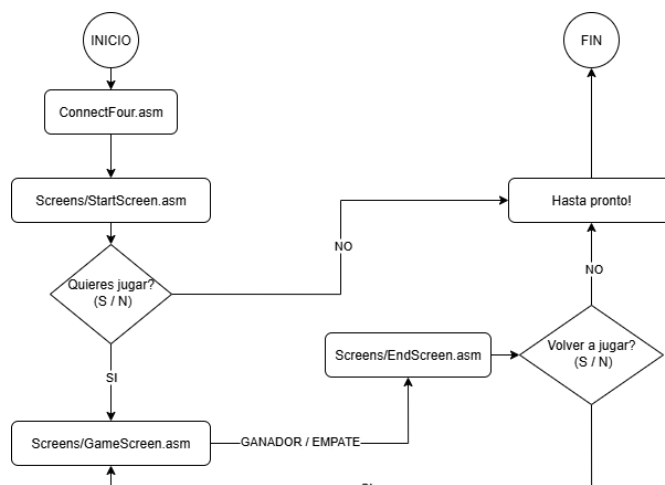


Figura 2-1: Flujo principal del programa

2.1 Resumen general

El flujo de ejecución del programa es un bucle de estado controlado por saltos (JP) entre diferentes pantallas:

1. **Inicio:** La ejecución comienza en ConnectFour.asm. Se deshabilitan las interrupciones (DI) y se inicializa el puntero de pila (LD SP, \$FFFF).

```

1  ; =====
2  ARCHIVO: ConnectFour.asm
3  ; USO: Archivo principal y punto de entrada (entry point) del juego "Conecta 4".
4  ; Define la configuración del ensamblador Z80, inicializa la CPU,
5  ; la pila (stack) y transfiere el control a la pantalla de inicio.
6  ; =====
7
8  DEVICE ZXSPECTRUM48
9  SLDOPT COMMENT WPMEM, LOGPOINT, ASSETION
10 ORG $8000
11
12 ; =====
13 PUNTO DE ENTRADA e INICIALIZACION
14 ; =====
15 BEGIN:
16 DI
17 LD SP, $FFFF ; Inicializamos la pila al final de la memoria para no chocar con la direccion ORG $8000 (por si acaso)
18 CALL CLEARSCR
19 JP START_SCREEN
20
21 ; =====
22 ; INCLUIDES DEL PROYECTO (ARCHIVOS .ASM)
23 ; =====
24 ; =====
25 ; 1. Datos (Variables y Constantes)
26 ; =====
27 INCLUDE "data/constants.asm"
28 INCLUDE "data/variables.asm"
29
30 ; =====
31 ; 2. Lógica del Juego
32 ; =====
33 INCLUDE "game/input.asm"
34 INCLUDE "game/logic.asm"
35
36 ; =====
37 ; 3. Gráficos (Rutinas de dibujo)
38 ; =====
39 INCLUDE "graphics/fichas.asm"
40 INCLUDE "graphics/portada.asm"
41 INCLUDE "graphics/pieza.asm"
42 INCLUDE "graphics/tablero.asm"
43 INCLUDE "graphics/utiles.asm"
44
45 ; =====
46 ; 4. Entrada/Salida (IO)
47 ; =====
48 INCLUDE "io/keyboard.asm"
49 INCLUDE "io/printat.asm"
50
51 ; =====
52 ; 5. Pantallas (Flujo del programa)
53 ; =====
54 INCLUDE "screens/common.asm"
55 INCLUDE "screens/StartScreen.asm"
56 INCLUDE "screens/GameScreen.asm"
57 INCLUDE "screens/EndScreen.asm"
58

```

Figura 2-2: Archivo ConnectFour.asm

2. **Pantalla de Inicio:** Se salta a START_SCREEN. Esta rutina muestra el título y la portada animada (PORTADA_DibujarPortada).
3. **Decisión:** StartScreen cede el control a COMMON_HANDLE_PLAY_RESPONSE.
 - Si el usuario pulsa 'S': Se salta a GAME_SCREEN.
 - Si el usuario pulsa 'N': Se salta a COMMON_MESSAGE_BYE_SCREEN (que finaliza con HALT).



Figura 2-3: Pantalla StartScreen.asm

4. **Pantalla de Juego:** GAME_SCREEN inicializa el estado del juego: limpia el tablero lógico (BOARD_ARRAY), reinicia las variables (CURRENT_COLUMN, GUARDAR_JUGADOR_ACTUAL) y dibuja el tablero gráfico (TABLERO_DibujarTableroCompleto).

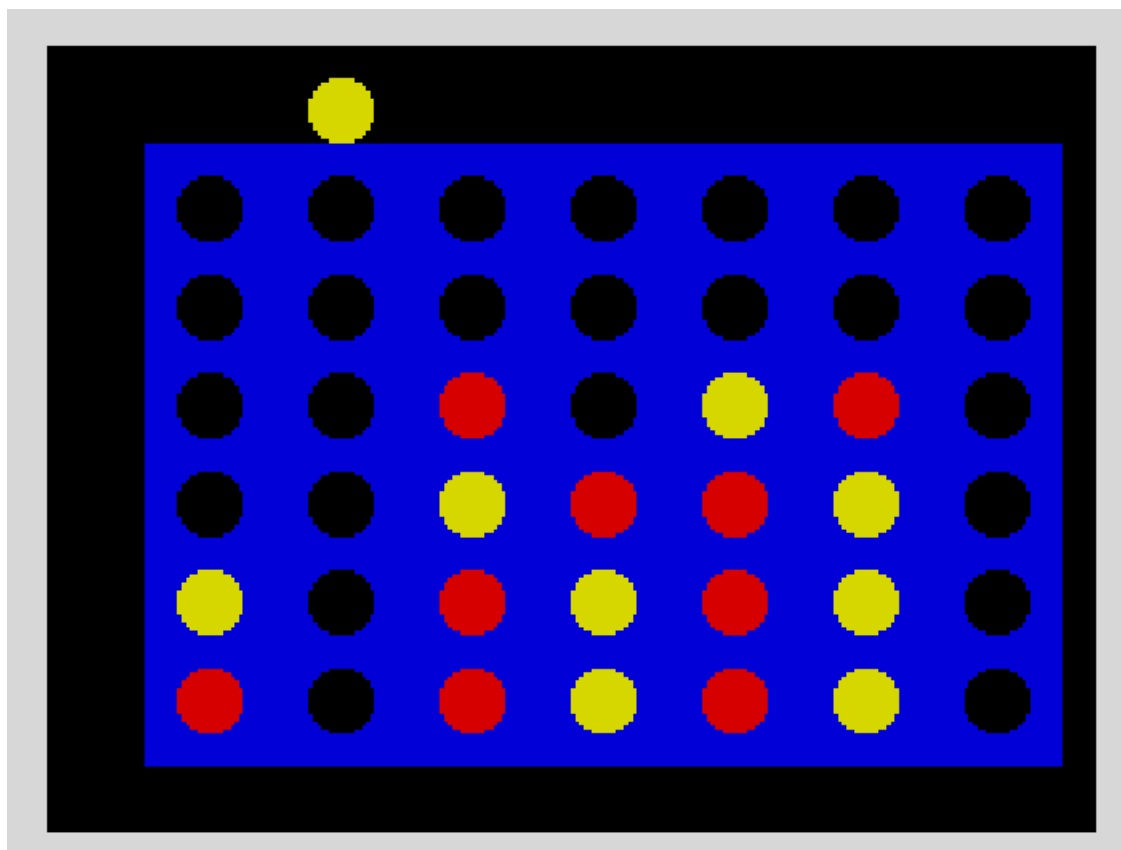


Figura 2-4: Pantalla GameScreen.asm

5. **Bucle de Juego:** GAME_SCREEN salta (JP) a KEYBOARD_ActivarInput_Keys. Este es el bucle principal del juego.
6. **Acción del Jugador:** El bucle de teclado (keyboard.asm) se ejecuta en un "bucle loco" (sin HALT). Gestiona el movimiento (Q/W para el Jugador 1, O/P para el Jugador 2) con un temporizador de 16 bits y espera la pulsación de ENTER.
7. **Colocar Ficha:** Al pulsar ENTER, se llama a COLOCAR_FICHA_EN_TABLERO. Esta rutina:
- Encuentra la fila vacía más baja en BOARD_ARRAY.
 - Ejecuta la animación de caída de ficha (controlada por DROP_ANIM_DELAY).

- Llama a CHECK_WIN para verificar si hay victoria.
 - Comprueba si hay empate (42 fichas).
 - Si no hay fin de partida, cambia de jugador (GUARDAR_JUGADOR_ACTUAL).
8. **Fin de Partida:** Si CHECK_WIN detecta una victoria o se llega al empate, input.asm llama a GAME_End, que a su vez salta a END_SCREEN.

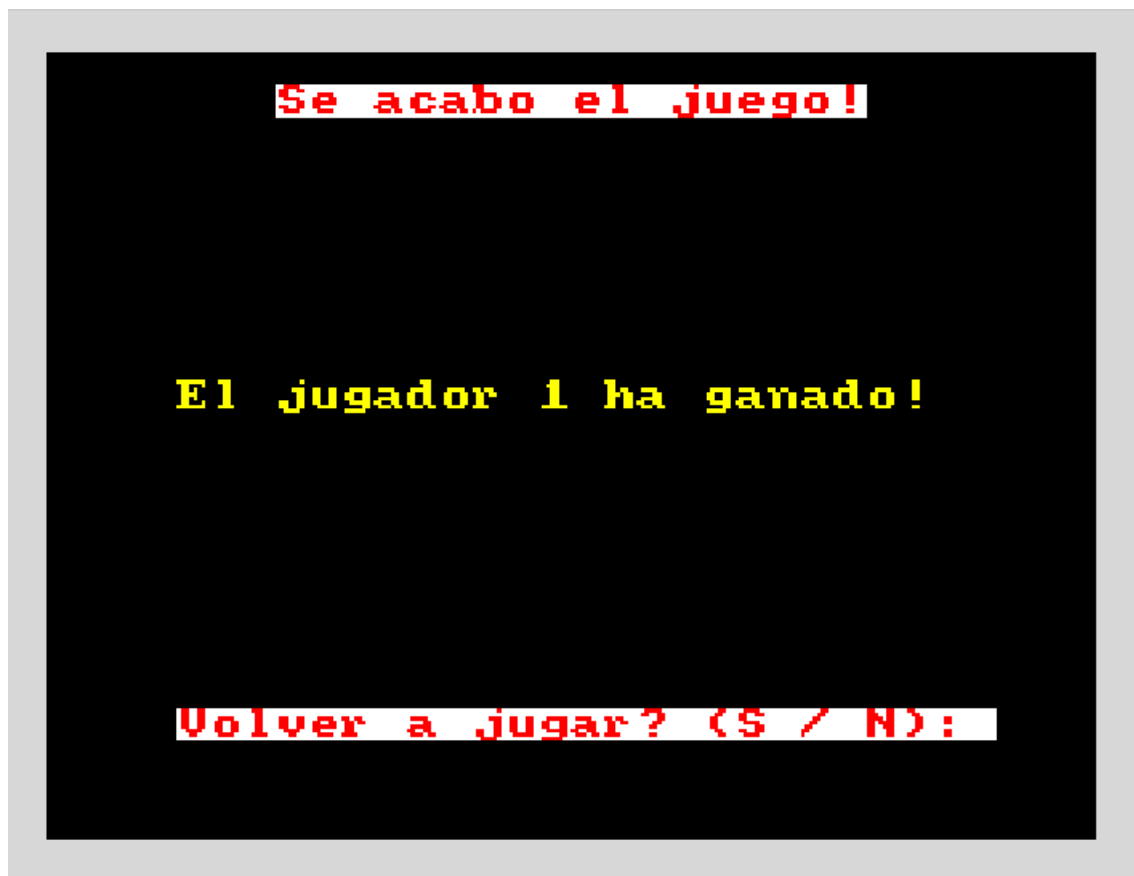


Figura 2-5: Pantalla EndScreen.asm: Jugador ganador

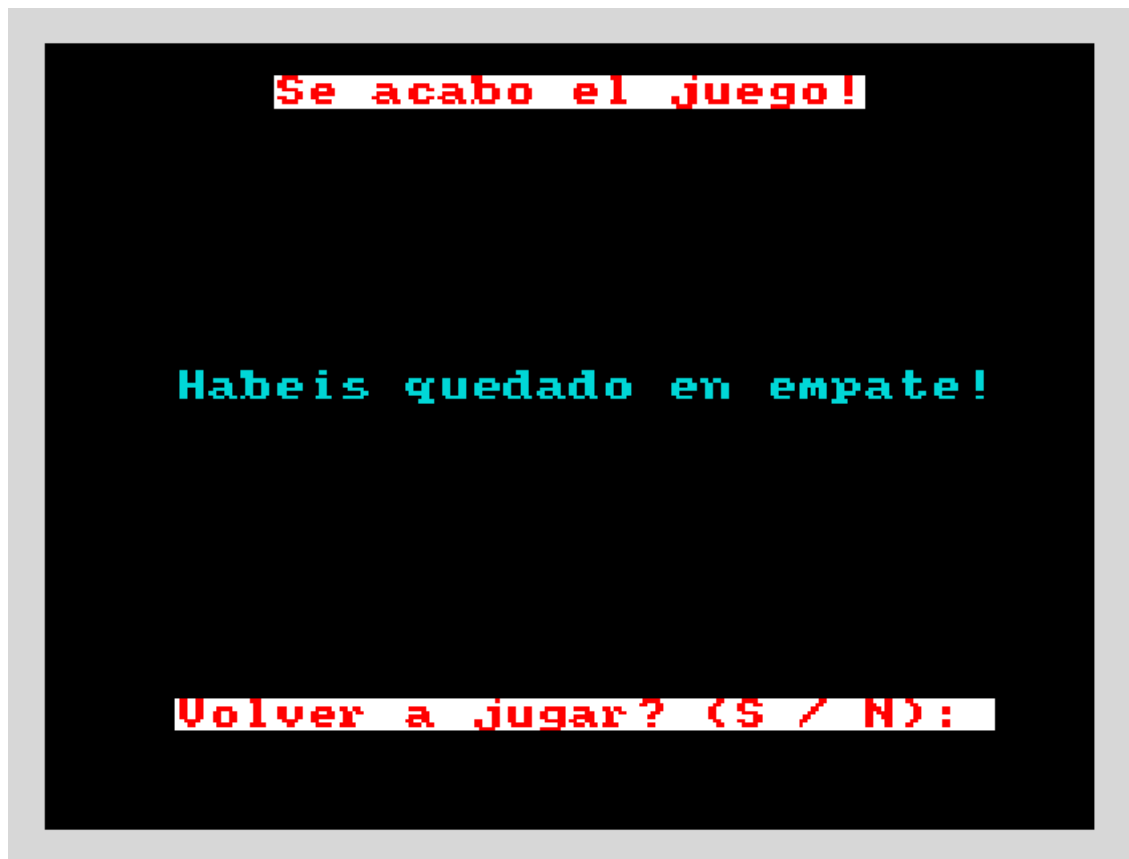


Figura 2-6: Pantalla EndScreen.asm: Empate entre jugadores

9. **Pantalla Final:** END_SCREEN muestra el resultado (leyendo GAME_OVER_REASON) y vuelve a ceder el control a COMMON_HANDLE_PLAY_RESPONSE para preguntar "Volver a jugar?". El ciclo vuelve al paso 3.



Figura 2-7: Pantalla common.asm: Mensaje de ¡Hasta pronto!

3 Rutinas principales

3.1 Listado de rutinas y descripción

- **ConnectFour.asm (BEGIN):** Punto de entrada. Configura la CPU (DI) y la pila (LD SP, \$FFFF) antes de saltar a START_SCREEN.
- **START_SCREEN:** Muestra la bienvenida y la portada gráfica (PORTADA_DibujarPortada). Delega la decisión de inicio a COMMON_HANDLE_PLAY_RESPONSE.
- **GAME_SCREEN:** Prepara el entorno para una nueva partida. Limpia BOARD_ARRAY, reinicia variables y dibuja el tablero (TABLERO_DibujarTableroCompleto).
- **END_SCREEN:** Pantalla de resultados. Muestra "Se acabó el juego!" y el mensaje de ganador o empate basándose en la variable GAME_OVER_REASON. Además, selecciona dinámicamente el color del texto (rojo o amarillo) para coincidir con el color del jugador ganador.
- **COMMON_HANDLE_PLAY_RESPONSE:** Rutina de navegación clave. Muestra un diálogo S/N y salta (JP) a GAME_SCREEN o a COMMON_MESSAGE_BYE_SCREEN según la respuesta.
- **KEYBOARD_ActivarInput_Keys:** Bucle de juego principal. Funciona como "busy-loop" (sin HALT), sondeando las teclas F, ENTER y las teclas de movimiento específicas del jugador (Q/W para Jugador 1, O/P para Jugador 2). Utiliza un temporizador de 16 bits (MOVE_COOLDOWN_TIMER) y una constante (MOVE_DELAY_FRAMES) para controlar la velocidad de repetición del movimiento lateral.
- **COLOCAR_FICHA_EN_TABLERO:** Lógica central del turno. Busca la fila disponible en BOARD_ARRAY, actualiza el tablero lógico, y ejecuta la animación de caída de la ficha, redibujando el hueco negro en cada paso para evitar artefactos visuales. Si la columna seleccionada se llena, esta invoca una rutina llamada UTIL_VISUAL_ERROR_FULL_COLUMN que emite un aviso visual al jugador parpadeando dos veces el borde de la pantalla en color rojo indicándole que no puede colocar la ficha ahí y retorna sin realizar la jugada para que la coloque en las columnas con huecos libres.
- **CHECK_WIN:** Lógica de victoria. Comprueba horizontal, vertical y ambas diagonales desde la última ficha colocada (LAST_ROW, LAST_COL). Usa el Carry Flag (CF) para reportar la victoria (CF=1).
- **FICHAS_PintarFicha_AjustadaTablero:** Rutina gráfica principal. Dibuja una ficha rellena de 16x16 píxeles (compuesta por 4 caracteres 8x8) usando las matrices _FICHA.
- **TABLERO_DibujarTableroCompleto:** Dibuja el fondo azul (TABLERO_RellenarRectAtributo) y luego dibuja 42 "huecos" (fichas negras) para formar la cuadrícula.
- **POSXY_CalcPixelAddr_4000 / POSXY_CalcAttrAddr_5800:** Funciones de bajo nivel esenciales que convierten coordenadas de carácter (H, L) en direcciones de memoria de píxeles y atributos de la pantalla del Spectrum.

4 Manual de uso

4.1 Requisitos previos

- Tener instalado VS Code y el archivo del compilador “sjasmpplus.exe”.
- En VS Code, configurar los archivos de dentro de la carpeta .vscode/ tasks.json y launch.json para que apunten a la ruta de “sjasmpplus.exe” y así poder ejecutar el emulador de ZX Spectrum 48K.

4.2 Como iniciar el juego

1. Compilar el archivo “ConnectFour.asm” para poder arrancar el programa y generar los archivos con extensiones .sld, .bin y .lst.
1. Aparecerá la pantalla de bienvenida con el título y una animación "C4".
2. El juego le preguntará "¿Quieres jugar? (S / N):".
 - Pulse **S** para comenzar la partida.
 - Pulse **N** para salir del programa.

4.3 Mecánicas del juego

- **Objetivo:** Conectar cuatro fichas de tu color (horizontal, vertical o diagonal).
- **Jugadores:** El Jugador 1 usa fichas **Rojas**. El Jugador 2 usa fichas **Amarillas**.
- **Controles en Partida:** las teclas de movimiento (Q, W, O y P) tienen repetición automática si se mantienen pulsadas, controlada por MOVE_DELAY_FRAMES.

Jugador 1 (Rojo):

- **Tecla Q:** Mover la ficha flotante hacia la izquierda.
- **Tecla W:** Mover la ficha flotante hacia la derecha.

Jugador 2 (Amarillo):

- **Tecla O:** Mover la ficha flotante hacia la izquierda.
- **Tecla P:** Mover la ficha flotante hacia la derecha.

Controles comunes de ambos jugadores:

- **Tecla ENTER:** Soltar la ficha en la columna seleccionada. Esto activará la animación de caída.
- **Tecla F:** Rendirse / Salir de la partida en curso.

- **Fin de Partida:**

- Cuando un jugador gana o el tablero se llena (empate), la partida termina.
- Se mostrará el resultado (Ganador 1, Ganador 2 o Empate).
- El juego preguntará "¿Volver a jugar? (S / N):".
- Pulse **S** para reiniciar el tablero y jugar de nuevo.
- Pulse **N** para ver el mensaje de despedida y terminar el programa.

5 Porcentajes de participación

Víctor Manuel Sanz Rubio → 50%

Roberto García Ramírez → 40%

Ignacio González de la Cruz → 10%

6 Bibliografía

[Todo el documento] Sánchez Mañes, José Manuel. TEMA 1: Introducción a la Arquitectura de Computadores.pdf. Lugar de publicación: Canvas UFV, 2025

[Todo el documento] Sánchez Mañes, José Manuel. TEMA 2: Arquitectura del Repertorio de Instrucciones (ISA).pdf. Lugar de publicación: Canvas UFV, 2025

[Todo el documento] Sánchez Mañes, José Manuel. TEMA 3: Organización del Computador.pdf. Lugar de publicación: Canvas UFV, 2025

[Todo el documento] Sánchez Mañes, José Manuel. TEMA 4: Rendimiento del Sistema.pdf. Lugar de publicación: Canvas UFV, 2025

[Todo el documento] Sánchez Mañes, José Manuel. TEMA 5: Optimizaciones.pdf. Lugar de publicación: Canvas UFV, 2025